DANCE: Dual-Side Agile N:M Sparse Compressed Digital CiM Accelerator for Efficient Compound AI

Zhonghao Chen*, Hongtao Zhong*, Jianhe Deng, Mulin Shi, Yongpan Liu, Huazhong Yang†, Xueqing Li†
Department of Electronic Engineering, LFET/BNRist/SKLSNC, Tsinghua University
*Equal Contributions; †Corresponding Email: xueqingli@tsinghua.edu.cn, yanghz@tsinghua.edu.cn

2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD) | 979-8-3315-1560-7/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICCAD66269.2025.11240835

***Abstract*—Compound AI systems showcase impressive performance and versatility compared to single AI models by combining large language models (LLMs) with various smaller expert models. The major bottleneck of compound AI lies in frequent data movement due to the massive parameters and dynamic routing mechanisms. Compute-in-Memory (CiM) has demonstrated great potential to mitigate the memory wall. However, constrained by the rigid array structure, existing CiM accelerators struggle to meet more general and diverse model compression demands of compound AI, such as fine-grained pruning for expert models and outlier-aware quantization for LLM-based router models. The lack of support for agile model compression hinders the deployment of compound AI systems on CiM accelerators.**

To fully unlock the potential of CiM in accelerating compound AI, we present DANCE, a dual-side N:M sparse compressed digital CiM architecture with cross-layer co-optimizations: (i) At the circuit level, DANCE introduces a customized set-associative selection circuit to extract N:M sparse patterns for both weights and activations, maintaining high parallelism; (ii) At the architecture level, DANCE explores a novel design paradigm that integrates fine-grained pruning and outlier-aware quantization into a unified N:M sparsity compression framework. Experimental results show that DANCE achieves up to 4.36× energy efficiency improvement with <1% accuracy loss for ResNet-18 on CIFAR-100, and up to 2.59× energy efficiency improvement with <0.5 perplexity increase for Llama-7B on WikiText-2, compared to the conventional digital CiM baseline.

***Index Terms*—compute-in-memory, compound AI, N:M sparsity, outlier-aware quantization, large language models.**

I. Introduction

Large language models (LLMs) with billions of parameters have shown considerable performance superiority over conventional deep neural networks (DNNs). However, the exponential growth in memory footprints and computational requirements, as predicted by the scaling law [1], impedes the economic deployment of LLMs. To mitigate this challenge, a practical solution is shifting from monolithic LLMs to compound AI systems [2]–[4], which employ LLMs as router models with various much smaller expert models tailored for downstream tasks, as shown in Fig. 1. This approach balances model performance and hardware efficiency, making it particularly appealing for resource-constrained devices. However, compound AI requires the LLMs as router models to select expert models, which still induces excessive memory access and huge computational demands.

Compute-in-Memory (CiM) is a promising approach towards higher efficiency for compound AI by integrating the matrix-vector multiplication (MVM) operations into memory to reduce data movement and improve parallelism [4]. In the early stage, CiM mainly focuses on analog-based approaches for better hardware efficiency [5]–[8]. However, due to process variations and noise, analog CiM suffers severe accuracy loss

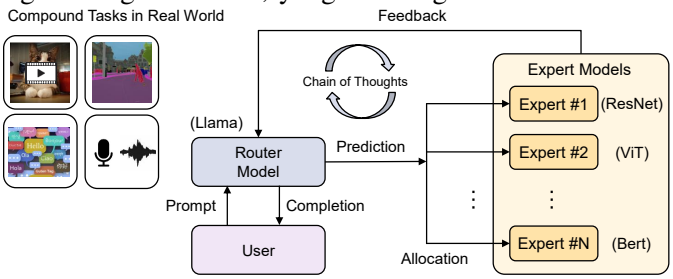


Fig. 1. Illustration of a typical compound AI system.

for complex DNN tasks. Recently, digital CiM has received significant focus due to its high precision and scalable energy efficiency improvement [9]–[15].

However, existing CiM designs still face critical challenges in accelerating compound AI. The first challenge is **limited flexibility for sparse pruning**. Constrained by the regular array structure, structured pruning, as shown in Fig. 2(a), is often utilized to improve performance and energy efficiency. However, most of the prior works [6], [15] only support one or few task-specific structured sparsity patterns with coarse granularity, which is not sufficient for compound AI with diverse model features. The second challenge is **severe accuracy loss during low-bit FP-INT alignment**. Prior floating-point (FP) CiM leverages the FP-INT pre-alignment to improve energy efficiency with negligible accuracy loss for conventional DNN models [11], [13], [14]. However, this method fails for LLMs due to more severe outlier-induced errors [16], [17], as depicted in Fig. 2(b). Besides, prior works lack integration of fine-grained pruning and outlier-aware quantization, which is critical for agile model compression in compound AI. For expert models, pruning has little effect on accuracy after fine-tuning sparse pretrained models to target tasks via transfer learning [18], [19]. However, for the less over-parametrized LLMs, pruning can induce large accuracy loss [20], [21]. In contrast, low-bit post-training quantization (PTQ) is a more cost-effective way to compress LLMs with negligible accuracy loss, especially when outliers are preserved in high-precision format [22]–[27].

To address these challenges, one intuitive solution is to develop a more general and CiM-friendly model compression strategy. N:M sparsity, an emerging semi-structured sparsity pattern, offers a promising approach to reconcile software flexibility and hardware efficiency. Its adoption in NVIDIA Ampere GPU architecture [28] has inspired remarkable progress in N:M sparsity related pruning algorithms [29]–[31] and DNN accelerators [32]–[34]. However, prior works mainly focus on single-side N:M sparsity, whereas the dual-side sparsity holds greater potential [35], [36]. Moreover, N:M sparse patterns can

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:45:59 UTC from IEEE Xplore. Restrictions apply.

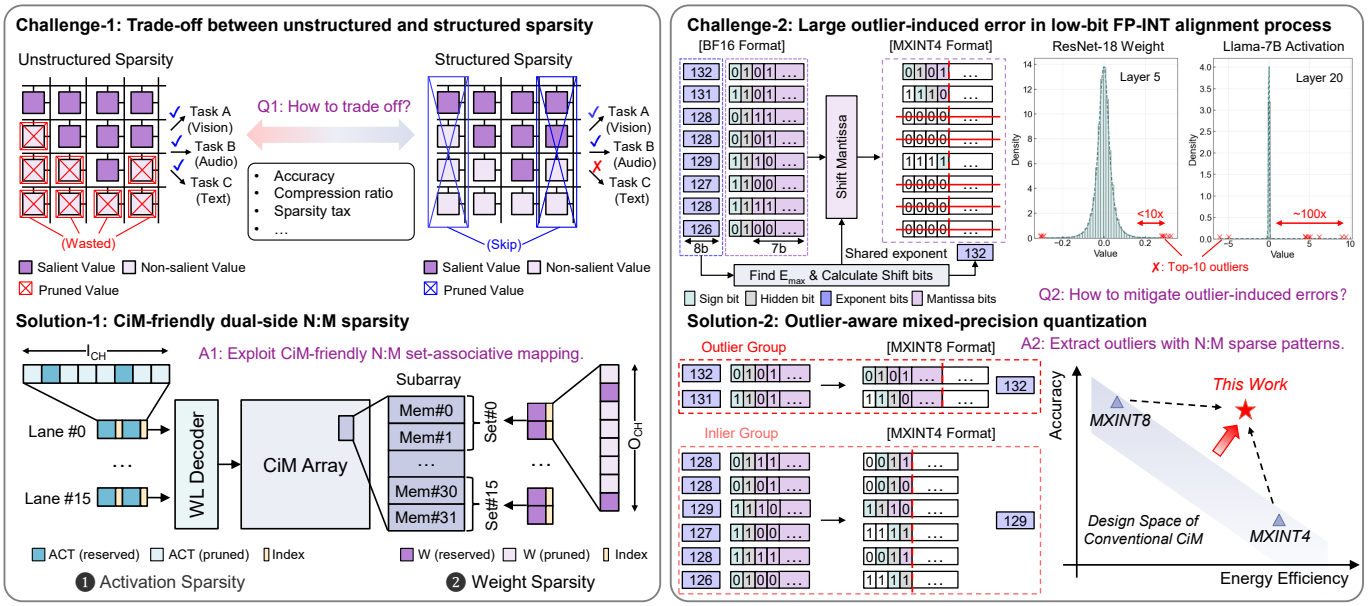


Fig. 2. Challenges of deploying compound AI workloads on CiM accelerators and the solutions of the proposed DANCE architecture.

also be used to extract outlier values in high-precision formats. These observations raise a central question: *Is it possible to design a dual-side N:M sparse compressed CiM accelerator to enable more agile model compression for compound AI?*

This work proposes DANCE to affirmatively answer this question, as shown in Fig. 2. To the best of our knowledge, DANCE is the first agile and scalable CiM architecture that integrates fine-grained pruning and outlier-aware quantization through a unified N:M sparse compression approach. Thus, DANCE satisfies more general model compression demands of compound AI. The key contributions are itemized as follows:

- We propose a customized set-associative N:M selection circuit with flexible mapping flow to support dual-side N:M sparse compression for both weights and activations.
- We utilize the N:M sparse pattern for outlier-aware quantization, which effectively extracts outliers and mitigates truncation errors during the FP-INT pre-alignment.
- We propose a digital SRAM CiM design based on the DANCE architecture, which adopts the Microscaling (MX) format [37] and integrates fine-grained pruning and outlier-aware quantization into a unified framework.
- We perform extensive evaluations on typical implementation options of DANCE. Results show that DANCE achieves up to 4.36× energy efficiency improvement with less than 1% accuracy loss for ResNet-18 on CIFAR-100, and up to 2.59× energy efficiency improvement with less than 0.5 perplexity increase for Llama-7B on WikiText-2 compared to the digital CiM baseline.

II. Background and Related Works

This section first introduces the basics of compound AI and the progress of existing CiM accelerators. Existing model compression methods, like N:M sparsity and outlier-aware quantization, are then reviewed to analyze prospects and challenges associated with their deployment on CiM accelerators.

A. Compound AI Systems

Compound AI systems integrate multiple interactive components to tackle complex problems beyond the capabilities of single AI models. Expert models are fine-tuned for specific tasks while router models process input prompts and dynamically select appropriate experts. This synergy improves problem-solving abilities and reduces system costs.

However, the dynamic interaction in compound AI results in frequent memory access and huge computational demands. Model compression techniques, such as pruning and low-bit quantization, are promising to alleviate this challenge. For expert models, pruning can boost hardware efficiency while maintaining high accuracy for downstream tasks. Prior works [18], [19] show that some sparse models, such as ResNet-50 with 90% sparsity pretrained on ImageNet, can catch up or even outperform some dense models in downstream tasks through transfer learning. For router models, maintaining text understanding and token generation capabilities is crucial to system stability. However, the LLM performance may be heavily degraded by pruning (e.g., just 50% weight pruning can significantly increase text perplexity [20], [21]). Therefore, low-bit post-training quantization techniques are more utilized with much smaller performance degradation of LLMs. For example, LLMs retain most of their language generation ability after 4-bit quantization with the outliers in high-precision format [16], [23]–[26]. Nevertheless, unifying pruning and outlier-aware quantization in a single architecture poses additional challenges for hardware designs, and prior CiM accelerators fail to achieve such an integration.

B. Compute-in-Memory

Compute-in-Memory is an emerging architecture to overcome the memory wall and accelerate high-parallel multiply-and-accumulate (MAC) operations in DNN workloads. Current CiM research can be categorized into analog CiM [5]–[8]

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:45:59 UTC from IEEE Xplore. Restrictions apply.

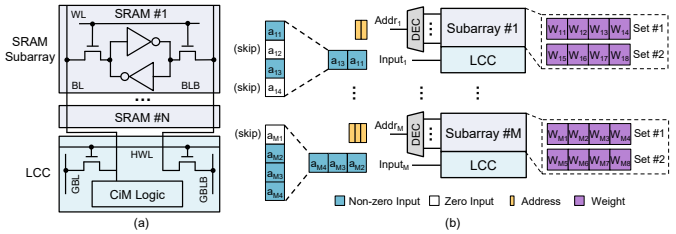


Fig. 3. Illustration of in-array CiM: (a) circuit diagram [5], including SRAM subarray and LCC; (b) Set-associative zero-skipping method [38].

and digital CiM [4], [9]–[15]. Compared to analog CiM, digital CiM achieves full-precision computation and technology-scalable energy efficiency improvements. Thus, recent works attempt to unify integer (INT) and floating-point (FP) operations within a single digital CiM macro. This advancement enables digital CiM to handle complex inference tasks and even training. However, digital CiM faces challenges due to the large area and power overhead of digital logic, like multipliers and adder trees. To tackle this, the LUT-based MAC technique is proposed in [10] to reduce the adder tree overhead. Besides, FP CiM requires additional exponent processing logic [4], [13], [14], which further increases digital logic overheads. To address this, pre-alignment based FP CiM has been proposed in [11], but this method results in truncation errors.

To alleviate the area overhead of large computation units, it is cost-effective to adopt the in-array CiM design paradigm [5], [13], [14], [38]. This method enables a large-area local computing cell (LCC) to be shared among multiple compact SRAM cells, as shown in Fig. 3(a). Moreover, the in-array CiM paradigm expands the design space. For example, ISSA [38] leverages the intrinsic set-associativity of in-array CiM to skip zero inputs, achieving more than 2× speedup, as depicted in Fig. 3(b). However, ISSA focuses on unstructured input sparsity, resulting in load imbalance issues.

C. Pruning and N:M Sparsity

Sparse pruning can improve computational efficiency and reduce memory footprints by skipping the operations with zero elements. As shown in Fig. 4(a), existing sparse pruning can be categorized into the unstructured sparsity [39] and the structured sparsity [40]. Unstructured sparsity achieves high compression ratios with minimum accuracy loss but incurs large sparsity tax due to indexing overhead and load imbalance issues [38], [41], making it less suitable for structured CiM arrays. Meanwhile, structured sparsity, which removes less important blocks, channels, or tokens in DNN models, aligns better with regular CiM arrays [6], [12], [15]. However, coarse-grained structured pruning is usually task-specific and fails to meet the diverse demands of compound AI.

N:M sparsity, where each block of M elements contains N nonzero elements [28], balances efficiency and accuracy by leveraging *intra-block* unstructured sparsity and *inter-block* structured sparsity. This pattern can be generated with dedicated training algorithms [28]–[31], [42]. Fig. 4(b) shows a spatial-multiplex N:M sparsity selection circuit using compressed sparse row (CSR) format adopted in NVIDIA Ampere GPU architecture. However, CSR-based weight compression

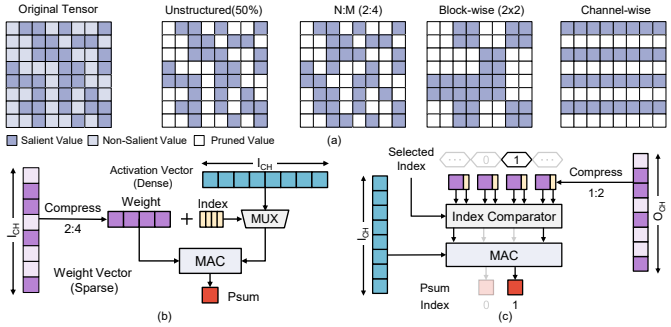


Fig. 4. (a) Sparsity patterns with different granularity; (b) Spatial-multiplex based N:M sparsity [28]; (c) Time-multiplex based N:M sparsity [34].

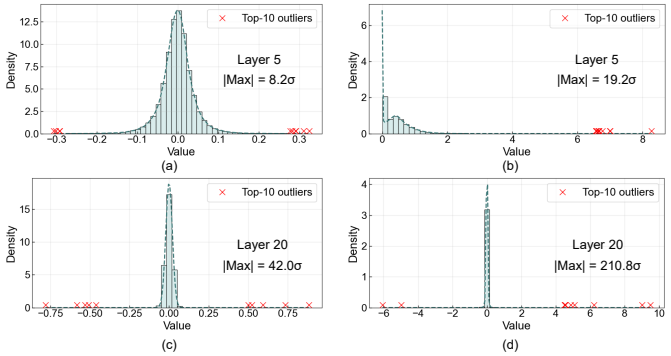


Fig. 5. Outlier distribution: (a) ResNet-18 weight; (b) ResNet-18 activation; (c) Llama-7B weight; (d) Llama-7B activation.

disrupts the CiM parallelism. Recently, [34] develops a CiM-friendly N:M weight sparsity with time-multiplexing and compressed sparse column (CSC) format, as shown in Fig. 4(c). However, this approach requires C_M^N cycles to traverse index combinations with a $\frac{M}{N}$ compression ratio, which becomes very time-consuming when $N > 1$. In addition, prior works mainly focus on single-side sparsity, whereas dual-side sparsity has greater potential to improve hardware efficiency. However, deploying dual-side N:M sparsity is very challenging for CiM accelerators, considering array-level constraints.

D. Quantization and Outlier Effects

Quantization is an effective method to represent DNN models with lower bitwidths. For example, quantizing a BF16 tensor into an INT4 tensor can be formulated as follows:

$$S = \frac{|X|_{max}}{2^{n-1}-1},\ X_q = round(\frac{X}{S}), \tag{1}$$

where X is the original BF16 tensor, X_q is the quantized INT4 tensor and S is the scale factor. Outliers significantly degrade resolution $(1/S)$, causing substantial accuracy loss in low-bit quantization. The limitation of per-tensor quantization inspires the development of per-block quantization. Microscaling (MX) [37] is an emerging data format where each block contains a shared exponent scale. MXINT is a subtype of MX format with three benefits: higher dynamic range for shared exponent, improved hardware efficiency via INT-based PEs, and easier format conversion with simple shift operations. Thus, MXINT better balances efficiency and accuracy for DNN accelerators. A multi-mode digital CiM accelerator supporting MXINT8 has demonstrated <0.5% accuracy loss

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:45:59 UTC from IEEE Xplore. Restrictions apply.

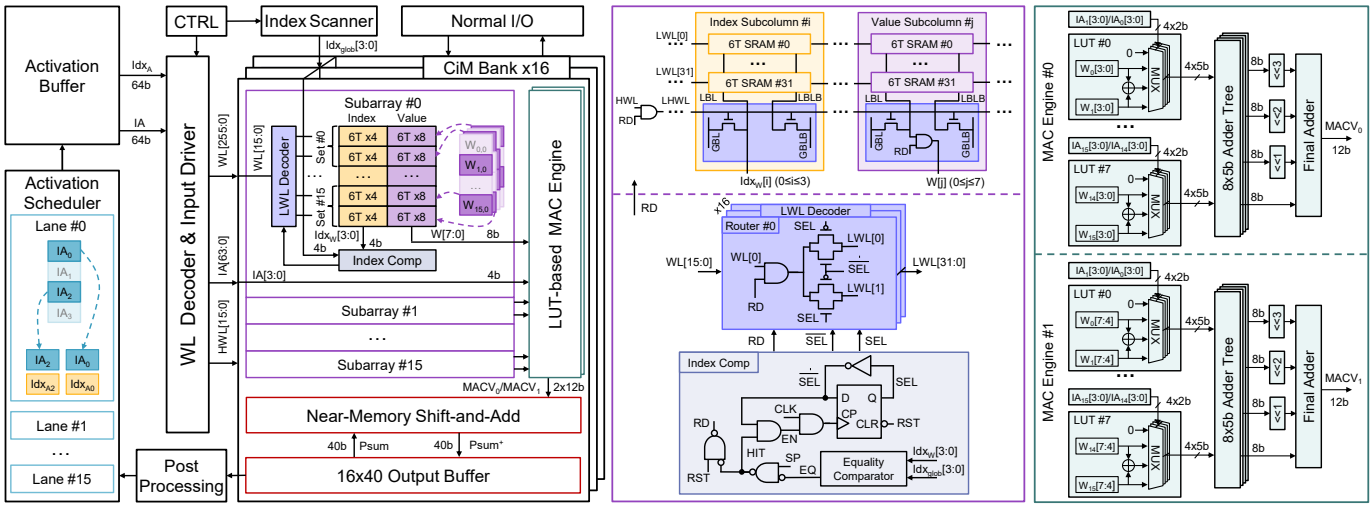


Fig. 6. Microarchitecture of the proposed DANCE core: (a) Digital CiM macro design based on DANCE architecture; (b) SRAM subarray schematic and index comparison logic with local WL/HWL gating; (c) LUT-based MAC engines supporting reconfigurable MAC operations.

on ImageNet [14]. However, MXINT still faces large outlier-induced errors when scaling to lower bits, like MXINT4 [26].

For LLMs, low-bit quantization is more challenging due to the emergence of much larger outliers, as shown in Fig. 5. To mitigate this, recent studies suggest that keeping outliers in high precision (e.g., FP16 or BF16) maintains accuracy. However, randomly distributed outliers complicate hardware deployment. For example, SpQR [22] extracts unstructured weight outliers with CSR format, which requires two 16-bit scalars for each outlier (a 16-bit global index and a 16-bit value) and induces large sparsity tax. Oltron [27] observes that most activation outliers fall within specific channels and retains salient channels in BF16 to improve hardware efficiency. However, this method omits fine-grained outlier patterns, resulting in large accuracy loss. OPAL [26] preserves four outliers for each 128 activations in BF16 to reduce truncation errors in MXINT4. However, OPAL requires heterogeneous FP/INT PEs with a fixed outlier ratio, hindering its flexibility for compound AI workloads. While many software algorithms and hardware architectures support outlier-aware quantization, it remains under-explored in existing CiM accelerators.

III. Proposed DANCE Architecture

This section presents the proposed DANCE architecture, including its top-level overview, sparse compression workflow, circuit-level details, and task-aware implementation options.

A. Overview

Fig. 6(a) illustrates the overall architecture of the SRAM-based digital CiM core in DANCE, which comprises 16 CiM banks and associated peripheral circuits, such as a WL decoder, input driver, normal I/O interface, index scanner, activation scheduler, and top controller. Each CiM bank consists of 16 subarrays, two LUT-based MAC engines similar to those in [10], and a near-memory shift-and-add block with a local output buffer. Each subarray with 32×12 foundry-rule 6T SRAM cells is divided into a value-subarray and an index-subarray, equipped with a shared index comparison logic detailed in Fig. 6(b).

Before MAC operations, weights and activations are pre-aligned based on MXINT format [37] with a group size of 256. During the MAC operations, 16 4-bit input activations are broadcast to CiM banks in each cycle. Within each subarray, only one row is selected by the WL decoder, and sends either an 8-bit weight or two 4-bit weights to the MAC engines. As shown in Fig. 6(c), each MAC engine performs 16 4-bit×4-bit MAC operations per cycle. The 2×12b partial MAC values (MACVs) in each CiM bank are delivered to the near-memory shift-and-add block. Finally, the accumulated partial sums are stored in the local output buffer.

Beyond vanilla MAC operations, DANCE supports fine-grained model compression by exploiting dual-side N:M sparsity. For weights, sparse patterns are obtained offline from pretrained models, and salient weights with their indices are stored in the CiM array. During the MAC operations, compressed weights are fetched when the stored index ($\mathrm{Idx_W}$) matches the broadcast global index ($\mathrm{Idx_{glob}}$). For activations, sparse patterns are dynamically identified with top-K selection logic in the activation scheduler. Salient activations with their indices are then fetched to the activation buffer, while masked activations are skipped. During the MAC operations, the indices are delivered to the WL decoder to activate specific rows in each subarray. This method enables masked input skipping with balanced workload across different input lanes.

B. Workflow of Dual-Side N:M Sparse Compression

The DANCE architecture also introduces a general workflow to deploy dual-side N:M sparse patterns, built upon the macro-level DANCE core design. This workflow compresses input activations and weights along input channels ($\mathrm{I_{CH}}$) and output channels ($\mathrm{O_{CH}}$), respectively, ensuring orthogonal optimizations to prevent potential bank conflicts.

First, a masked input skipping approach is proposed to exploit N:M activation sparsity, as outlined in Algorithm 1. In each set of M_i input activations, only the preserved N_i values ($A[B, N_i]$) and their indices ($I[B, N_i]$) are streamed to CiM banks. This approach avoids the load imbalance issue in

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:45:59 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 In-lane Masked Input Skipping

Define: $B/N_i/M_i$: Number of batches/tokens and the adopted N:M input activation sparsity pattern.
Define: index/input activation matrix $I[B, N_i]/A[B, N_i]$ that records indices/activations prefetched in the buffer.
Define: streaming input activation value A_S and target row address I_S.
for $i = 1 \rightarrow B$ **do**
  // Dimension: Token or Batch
  for $j = 1 \rightarrow N_i$ **do**
    // Dimension: Input channel
    Let $I_S \leftarrow I[i, j]$
    Let $A_S \leftarrow A[i, j]$
    Select target row according to I_S;
    Stream A_S to CiM banks.
  end for
end for

Algorithm 2 In-subarray Compressed Weight Unrolling

Define: $S/N_w/M_w$: Number of sets and the adopted N:M weight sparsity pattern in the subarray.
Define: index/weight matrix $I[S, N_w]/W[S, N_w]$ that records indices/weights stored in the subarray.
Define: readout weight value W_R.
for $i = 1 \rightarrow S$ **do**
  // Dimension: Input channel
  Let $c \leftarrow 1$;
  for $j = 1 \rightarrow M_w$ **do**
    // Dimension: Output channel
    if $I[i, c] == j$ **then**
      Let $W_R \leftarrow W[i, c]$;
      Let $c \leftarrow c + 1$;
    else
      Let $W_R \leftarrow 0$;
    end if
    Fetch W_R to the MAC Engine.
  end for
end for

ISSA [38] and achieves up to $\frac{M_i}{N_i}\times$ speedup. On the hardware side, the activation scheduler can be shared across different CiM cores, effectively reducing hardware overhead. Therefore, the DANCE core is designed to support up to 8-way set-associative activation compression for better accuracy.

Second, a compressed weight unrolling method is developed to ensure correct MAC operations and reduce readout power, as shown in Algorithm 2. Compressed weights ($W[S, N_w]$) are stored in CiM banks with their indices ($I[S, N_w]$) using N_w-way set-associative mapping. During the MAC operations, M_w possible indices are traversed via a loop function; weights (W_R) are delivered to the MAC engine only when their indices ($I[i, c]$) match the traversed index (j); otherwise, they are skipped to save energy. This method reduces the number of index traverse cycles from $C_{M_w}^{N_w}$ to M_w compared with the prior work [34], avoiding throughput drop when $N_w > 1$. In terms of hardware implementation, Algorithm 2 requires a $\lceil \log_2 N_w \rceil$-bit counter with adder logic in each subarray. Besides, each set needs an N_w:1 demultiplexer (DMUX) for the local WL decoder. Since 2:M sparsity is enough to maintain accuracy for existing N:M weight pruning algorithms [29], [30], the proposed DANCE core adopts 2-way set-associative weight compression to minimize hardware overhead.

C. Circuit-Level Details of Set-Associative N:M Selection

To implement Algorithm 1 and Algorithm 2, a novel dual-side set-associative N:M selection circuit is proposed, which includes O_{CH}-wise weight compression in CSC format and I_{CH}-wise input activation compression in CSR format. Through algorithm-circuit co-design, the DANCE core supports up to 1:16 weight/activation compression ratio.

First, the activation scheduler and WL decoder are utilized for masked input skipping with N:M spatial-multiplexed activation sparsity, as shown in Fig. 6(a). The activation scheduler filters pruned activations according to the preset N:M ratio and generates indices (Idx_A) to activate specific rows in each subarray. As detailed in Fig. 7(a), the customized WL decoder

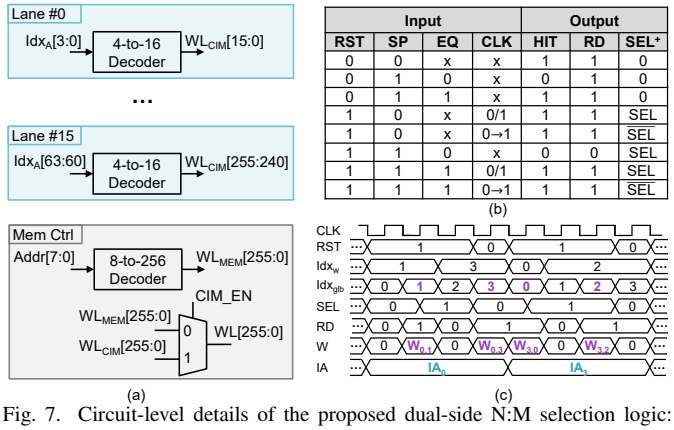


Fig. 7. Circuit-level details of the proposed dual-side N:M selection logic: (a) Customized WL decoder design; (b) Truth table of the index comparator; (c) Timing diagram example of 2:4 weight/activation sparsity.

comprises 16 4-to-16 decoders for masked input skipping and an 8-to-256 decoder for normal memory function. Besides, a MUX-based logic reconfigures the WL decoder between CiM mode and memory mode. The modified WL decoder is shared by multiple CiM banks, ensuring low hardware overheads.

Second, a local index comparison logic is introduced in each subarray to support 2-way set-associative N:M sparse weight compression and time-multiplexed unrolling, as depicted in Fig. 6(b). The index comparator includes an equality comparator to compare the weight index (Idx_W) with the traversed index (Idx_{glob}), a DFF to store a 1-bit local WL address and clock-gating logic. The local WL decoder is composed of 16 routers, each equipped with an AND gate for clock-gating and a transmission-gate based 2:1 DMUX. The index comparator generates 3 1-bit signals (RD, SEL, and $\overline{SEL}$) to locate the target row in each set. Fig. 7(b) shows the truth table of the index comparator. This block requires several global control signals (SP, CLK, and RST), where SP is the enable signal of sparsity mode, while the CLK and RST are used

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:45:59 UTC from IEEE Xplore. Restrictions apply.

to update or clear the state of DFF (SEL). Before the first weight access, RST resets the DFF, which also sets RD to 1. The equality comparator then compares Idx_W with Idx_{glob} each cycle. Once Idx_W matches Idx_{glob}, the HIT value is set to 1, which updates the SEL value and triggers the next weight readout. Otherwise, the weight value is gated with local WL/HWL turned off to ensure correct computation and reduce readout power. While primarily designed for 2:M sparsity, the proposed index comparison logic is also compatible with dense mode and 1:M sparsity. In dense mode, setting SP to 0 and RST to 1 enables weight reading each cycle. For 1:M sparsity, CLK remains disabled until starting to unroll a new coming weight set.

Fig. 7(c) shows a timing diagram example of deploying 2:4 weight/activation sparsity. In this example, DANCE achieves 2× throughput and 2× effective storage density, compared to dense mode. To unroll the compressed weights, four cycles are needed to traverse all output channels with the same selected input. The timing waveform also shows that the readout weights are gated until Idx_{glob} matches Idx_W, which ensures correct computation and reduces power.

D. Reconfigurable Near-Memory MACV Accumulation Flow

The proposed DANCE architecture supports quad-mode bitwidth configurations (W4A4, W4A8, W8A4, and W8A8) through spatial and temporal concatenation of MACVs. Fig. 8 illustrates the reconfigurable MACV accumulation process for these configurations. Each MAC engine performs 16 4-bit×4-bit MAC operations, producing a 12-bit MACV, as shown in Fig. 6(c). For 8-bit weight mode, two adjacent 12-bit MACVs are merged, as depicted in Fig. 8(c)-(d). Besides, for 8-bit input mode, two consecutive 12-bit MACVs from the same MAC engine can be combined, as shown in Fig. 8(b) and (d).

Besides, applying dual-side N:M sparsity to the DANCE core requires up to 16× registers for MACV accumulation since the time-multiplexed weight sparsity requires switching output channels to traverse all possible indices. To mitigate the register overhead, we propose a near-memory MACV accumulation scheme, where a local output buffer of 16×40 foundry 6T SRAM cells temporally stores partial sums of all possible output channels in each CiM bank. Given that MAC operations consume much more time than output buffer read/write operations, we double the output buffer's clock frequency, which enables its read/write operations to be completed within one single MAC cycle. This method prevents large register overhead or frequent global buffer access while maintaining CiM parallelism.

E. Task-Aware Implementation Options

Moreover, DANCE can be configured to support both fine-grained pruning and outlier-aware quantization, significantly extending the design space for CiM accelerators. This flexibility stems from the proposed dual-side N:M compression strategy and the reconfigurable CiM circuit design, which is highly versatile for compound AI systems. Fig. 9 presents two task-aware implementation options based on the DANCE core.

The Option-I, shown in Fig. 9(a), directly deploys dual-side N:M sparsity for pruning. Compressed weights and activations

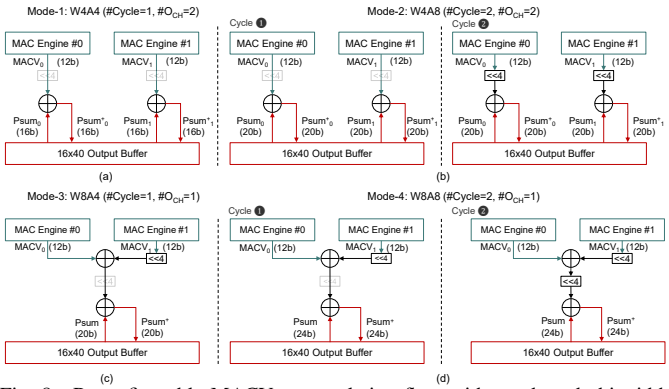


Fig. 8. Reconfigurable MACV accumulation flow with quad-mode bitwidth configurations: (a) W4A4; (b) W4A8; (c) W8A4; (d) W8A8.

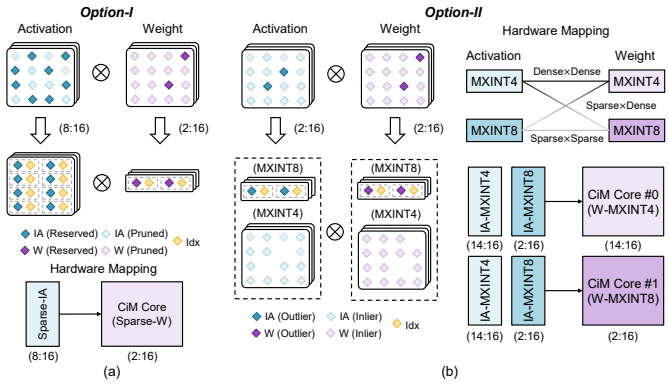


Fig. 9. Options of DANCE architecture for different scenarios with top-view examples: (a) option-I: N:M pruning flow for expert models; (b) Option-II: N:M outlier-aware quantization flow for LLM-based router models.

are mapped to a CiM core for the MAC operations. This configuration supports efficient model pruning with flexible compression ratios (from 2:4 to 2:16) for diverse expert models in compound AI. Besides, accuracy can be effectively recovered by fine-tuning, especially for transfer learning scenarios.

The Option-II, depicted in Fig. 9(b), uses the N:M sparsity to extract local outliers. Activation and weight matrices are decomposed into dense inlier matrices (MXINT4) and sparse outlier matrices (MXINT8) with index matrices. The original matrix multiplication (MM) is split into four sub-MMs, which are executed by CiM cores with full utilization supported by the reconfigurable MACV accumulation flow. This approach efficiently supports post-training outlier-aware quantization for LLM-based router models in compound AI systems.

IV. Evaluation and Discussion

A. Experimental Setup

Software Setup: To comprehensively evaluate the effectiveness of DANCE in terms of flexibility and scalability, the software experiments include the following scenarios:

(i) *N:M pruning for expert models.* We first apply various N:M pruning ratios on a dense ResNet-18 model pretrained on ImageNet. We obtain the accuracy and sparse models after 60-epoch fine-tuning using SR-STE [29]. Then we fine-tune these sparse models on CIFAR-100 with SR-STE for another 60 epochs to assess their performance in sparse transfer learning.

(ii) *N:M outlier-aware quantization for router models.* We validate the N:M outlier-aware quantization method on

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:45:59 UTC from IEEE Xplore. Restrictions apply.

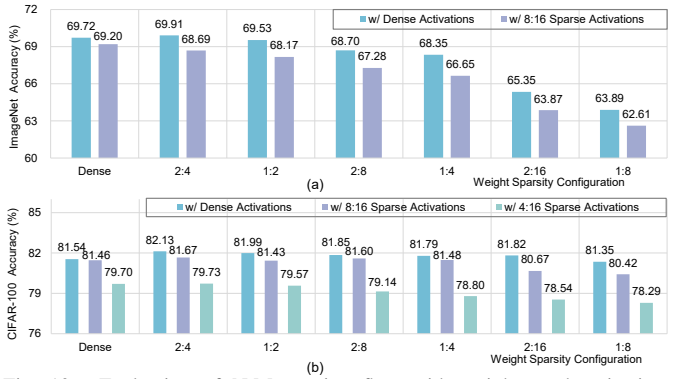


Fig. 10. Evaluation of N:M pruning flow with weights and activations quantized in MXINT8: (a) ResNet18 on ImageNet; (b) ResNet18 on CIFAR-100.

WikiText-2 with pretrained LLMs from Huggingface. We also use Microsoft's MX emulation library [43] to evaluate the proposed quantization scheme with a group size of 256.

Hardware Setup: The CiM core, based on the DANCE architecture, is implemented in a commercial 28nm CMOS process to evaluate its area, power, and throughput. Digital components, including the MAC engine and shift-and-add block, are synthesized using Design Compiler. Other components, such as the CiM subarray and output buffer, are evaluated via transistor-level layout implementation.

Baseline: We compare the proposed DANCE core with a typical digital CiM baseline from [10]. For fairness, the baseline is implemented with the in-array CiM paradigm in [5], where the LCC is shared among 32 SRAM rows per subarray. Macro-level performance is evaluated with the same CiM subarrays and peripheral components as the baseline, except for the additional sparsity compression logic.

B. Software Evaluation

Option-I: N:M Pruning Flow: Fig. 10(a) shows the top-1 accuracy for ResNet-18 on ImageNet after applying various N:M sparse pruning ratios and fine-tuning. Compared with the dense mode, the proposed N:M pruning can achieve a 2× effective reduction of the model parameters and a 2× speedup with only about 1% accuracy loss at 50% weight and 50% activation sparsity. Besides, the throughput and effective memory density can be further improved by applying more aggressive sparse patterns. At the same sparsity level, a larger N ensures better accuracy due to more diverse N:M mask combinations. Thus, DANCE can achieve 0.35%∼1.46% accuracy improvement over [34], by supporting 2:M weight sparsity, whereas the latter only supports 1:M weight sparsity.

Fig. 10(b) presents the top-1 accuracy for ResNet-18 on CIFAR-100 after sparse transfer learning. Results indicate that the N:M sparse models can catch up or even outperform the original dense model in transfer learning scenarios. For instance, with 2:16 weight sparsity, DANCE achieves 81.82% accuracy, exceeding the dense mode by 0.28%. Even after introducing additional 4:16 activation sparsity, DANCE still achieves 78.54% accuracy. This demonstrates the potential of N:M pruning flow of DANCE to accelerate expert models on downstream tasks via sparse transfer learning.

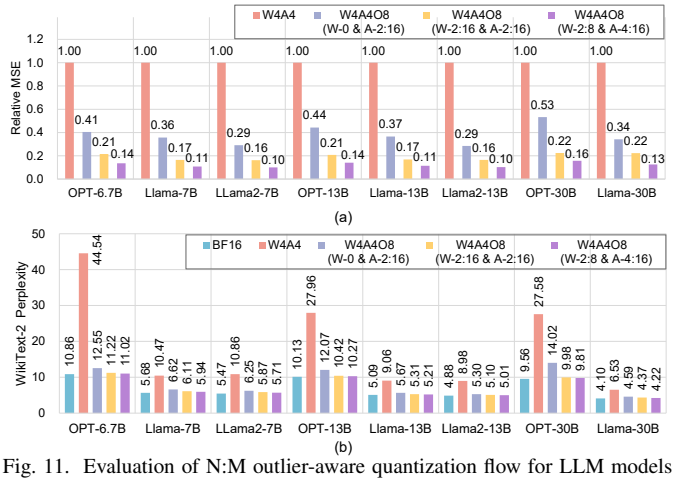


Fig. 11. Evaluation of N:M outlier-aware quantization flow for LLM models on WikiText-2: (a) Average output MSE results; (b) Perplexity results.

Option-II: N:M Outlier-Aware Quantization Flow: In Fig. 11(a), we evaluate the effectiveness of preserving local outliers in MXINT8 by calculating the average mean squared error (MSE) of output tensors between quantized models and original BF16 models. Results show that the MSE can be significantly suppressed by preserving outliers. For instance, the MSE is reduced by 7.09∼9.74× across various LLMs with 2:8 sparse weight outliers and 4:16 sparse activation outliers.

Fig. 11(b) shows perplexity results on WikiText-2 across different quantization schemes. Lowering bitwidth to MXINT4 induces up to 33.9 perplexity increase. In contrast, DANCE maintains less than 0.5 perplexity increase across various LLMs by preserving 2:16 sparse local weight and activation outliers in MXINT8 with other weights and activations quantized in MXINT4. Moreover, by adjusting the local outlier ratio according to real application demands, DANCE achieves a better accuracy-efficiency trade-off compared with the uniform quantization scheme in conventional CiM accelerators.

C. Hardware Evaluation

Overhead Analysis: Fig. 12(a) presents the power and area breakdown of the DANCE core. The MAC logic (LUT, adder tree, and shift-and-add block) dominates power consumption and area. Meanwhile, the additional index SRAM cells, index comparison logic, and other sparse compression related logic in the DANCE core account for only 13.8% power consumption and 18.8% area, proving the low hardware implementation cost of the proposed dual-side N:M sparsity.

The DANCE core adopts 2-way set-associativity for weight compression to balance hardware efficiency and software accuracy. To evaluate this trade-off, we also analyze the hardware cost and accuracy improvement associated with higher set-associativity for weights (N_w). As shown in Fig. 12(b), further increasing weight set-associativity improves little accuracy but significantly increases higher hardware overhead.

Fig. 12(c) validates the effectiveness of the proposed near-memory shift-and-add method to mitigate the register overhead from output channel switching. Compared with a naive implementation with 16 register groups, our method achieves 12.1× power and 7.9× area reduction. This optimization avoids ex-

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:45:59 UTC from IEEE Xplore. Restrictions apply.

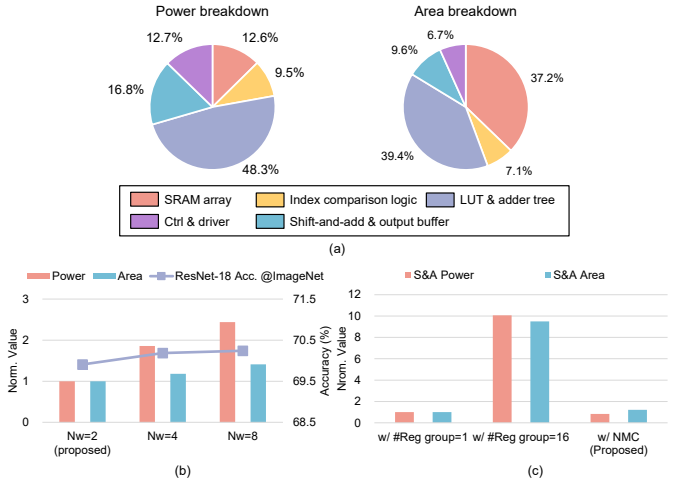


Fig. 12. Hardware evaluation and overhead analysis: (a) Macro-level power and area breakdown; (b) Design space analysis of N_w; (c) Analysis of register overheads and the proposed near-memory MACV accumulation scheme.

cessive register costs and ensures scalable hardware efficiency improvements with more aggressive sparse compression.

Macro-Level Improvements and Comparison: Fig. 13 shows the performance improvements across various hardware configurations. The N:M pruning flow (Option-I) achieves up to 4× speedup and 2.58∼8.18× energy efficiency improvement with 2:4∼2:16 weight sparsity and 8:16∼4:16 activation sparsity, compared with the dense mode, as shown in Fig. 13(a). The N:M outlier-aware quantization flow (Option-II) achieves up to 3.56× speedup and 2.16∼3.35× energy efficiency improvement with 0∼2:8 weight outlier ratio and 2:16∼4:16 activation outlier ratio, compared to the uniform W8A8 quantization scheme, as shown in Fig. 13(b).

Table I summarizes the macro-level comparison between the proposed DANCE core and the digital CiM baseline. DANCE supports various configurations and achieves up to 3.3× higher area efficiency, 7.1× higher energy efficiency, compared to the digital CiM baseline with only 23.1% area overhead.

D. More Discussions and Future Works

Flexibility: DANCE demonstrates the feasibility of deploying dual-side N:M sparse pattern in CiM accelerators with two task-aware implementation flows (Option-I and Option-II). Option-I targets sparse pruning on expert models, while Option-II is designed for outlier-aware quantization on LLM-based router models. This design enables DANCE to meet diverse model compression demands of compound AI.

Scalability: Implementing the proposed dual-side sparse compression scheme requires additional sparse compression logic. The sparse compression logic accounts for only <20% of macro-level power and area while achieving up to 1:16 compression ratio for weights and activations. Besides, the proposed near-memory MACV accumulation flow prevents large register overhead, enabling scalable efficiency improvements towards aggressive sparse compression.

Further Extensions: For Option-I, DANCE mainly focuses on deploying uniform N:M sparse patterns. Future exploration can involve the layer-wise or block-wise mixed N:M sparsity [30], [42], which is also promising to achieve compara-

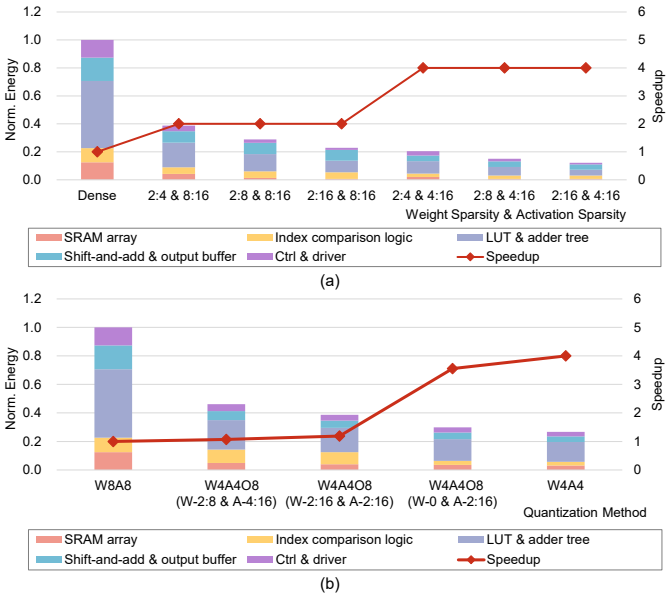


Fig. 13. Energy reduction and speedup across various hardware configurations: (a) N:M pruning flow; (b) N:M outlier-aware quantization flow.

TABLE I
MACRO-LEVEL PERFORMANCE COMPARISON

Metrics	Digital CiM Methodologies						
	Baseline[1]		*DANCE Option-I*		*DANCE Option-II*		
Process	28nm CMOS						
Voltage supply (V)	0.9						
Macro area (mm^2)	0.058		0.072				
Precision (MXINT)	W4A4	W8A8	W4A4	W8A8	W4A4	W8A8	W4A4O8[4]
Energy efficiency (TOPS/W)	37.5[2]	10.1[2]	32.7[2]-267.0[3]	8.7[2]-71.4[3]	32.7[2]	8.7[2]	8.7-29.2[5]
Area efficiency (TOPS/mm^2)	5.2[2]	1.3[2]	4.3[2]-17.1[3]	1.1[2]-4.3[3]	4.3[2]	1.1[2]	1.1-3.8[5]
Dual-side N:M pruning support	No		Yes		No		
Outlier-aware quantization support	No		No		Yes		

[1] In-array CiM design w/o N:M sparsity, adopting LUT-based MAC design in [10] shared by 32 SRAM rows each subarray. [2] Dense mode, no sparsity.
[3] Evaluated at 2:16 weight sparsity & 4:16 activation sparsity. [4] MXINT4 for inlier quantization, and MXINT8 for outlier quantization.
[5] Evaluated at dense mode, with 2:16 activation outlier in MXINT8, and other activations and weights in MXINT4.

ble accuracy to unstructured sparsity. Hierarchical structured sparsity (HSS) [44] also shows great potential to further reduce sparsity tax with flexible sparsity support. For Option-II, DANCE adopts the magnitude-based metric for outlier selection. Recent studies [16], [22] have explored sensitivity-based metrics to enhance accuracy, which is also helpful for DANCE. Integrating these algorithmic innovations with the DANCE architecture via system-level co-design provides a promising avenue for the evolution of future CiM accelerators.

V. CONCLUSION

This work introduces DANCE, an agile digital CiM architecture, to satisfy diverse model compression demands for compound AI. With CiM-friendly dual-side N:M sparse compression, DANCE unifies fine-grained pruning and outlier-aware quantization into a single framework. Experimental results show that DANCE effectively broadens the design space, boosting hardware efficiency while maintaining decent accuracy. Thus, DANCE offers an enhanced efficiency-accuracy Pareto frontier, compared to the existing CiM accelerators.

ACKNOWLEDGMENT

This work is supported in part by NSFC (92264204, U24B6015 and U21B2030) and National Key R&D Program of China (2024YFB3214500).

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:45:59 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] J. Kaplan *et al.*, "Scaling Laws for Neural Language Models," *arXiv preprint arXiv:2001.08361*, 2020.

[2] M. Zaharia *et al.*, "The Shift from Models to Compound AI Systems," https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems/, 2024.

[3] R. Prabhakar *et al.*, "Sambanova SN40L: Scaling the AI Memory Wall with Dataflow and Composition of Experts," in *2024 57th International Symposium on Microarchitecture (MICRO)*, 2024, pp. 1353–1366.

[4] Z. Yue *et al.*, "14.4 A 51.6 TFLOPs/W Full-Datapath CIM Macro Approaching Sparsity Bound and $< 2^{-30}$ Loss for Compound AI," in *2025 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 68, 2025, pp. 1–3.

[5] X. Si *et al.*, "A Local Computing Cell and 6T SRAM-based Computing-in-Memory Macro with 8-b MAC Operation for Edge AI Chips," *IEEE Journal of Solid-State Circuits*, vol. 56, no. 9, pp. 2817–2831, 2021.

[6] J. Yue *et al.*, "An Energy-Efficient Computing-in-Memory NN processor With Set-Associate Blockwise Sparsity and Ping-Pong Weight Update," *IEEE Journal of Solid-State Circuits*, vol. 59, no. 5, pp. 1612–1627, 2023.

[7] H. Wang *et al.*, "A Charge Domain SRAM Compute-in-Memory Macro With C-2C Ladder-Based 8-bit MAC Unit in 22-nm FinFET Process for Edge Inference," *IEEE Journal of Solid-State Circuits*, vol. 58, no. 4, pp. 1037–1050, 2023.

[8] K. Yoshioka, "34.5 A 818-4094TOPS/W Capacitor-Reconfigured CIM Macro for Unified Acceleration of CNNs and Transformers," in *2024 IEEE International Solid-State Circuits Conference (ISSCC)*. IEEE, 2024, pp. 1–3.

[9] Y.-D. Chih *et al.*, "16.4 An 89TOPS/W and 16.3 TOPS/mm^2 All-digital SRAM-based Full-precision Compute-in-Memory Macro in 22nm for Machine-learning Edge Applications," in *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 64, 2021, pp. 252–254.

[10] C.-F. Lee *et al.*, "A 12nm 121-TOPS/W 41.6-TOPS/mm^2 All Digital Full Precision SRAM-based Compute-in-Memory with Configurable Bit-width For AI Edge Applications," in *2022 IEEE Symposium on VLSI Technology and Circuits*, 2022, pp. 24–25.

[11] F. Tu *et al.*, "ReDCIM: Reconfigurable Digital Computing-In-Memory Processor With Unified FP/INT Pipeline for Cloud AI Acceleration," *IEEE Journal of Solid-State Circuits*, vol. 58, no. 1, pp. 243–255, 2022.

[12] F. Tu, Z. Wu, Y. Wang, L. Liang, L. Liu, Y. Ding *et al.*, "TranCIM: Full-Digital Bitline-Transpose CIM-based Sparse Transformer Accelerator With Pipeline/Parallel Reconfigurable Modes," *IEEE Journal of Solid-State Circuits*, vol. 58, no. 6, pp. 1798–1809, 2022.

[13] P.-C. Wu *et al.*, "An Integer-Floating-Point Dual-Mode Gain-Cell Computing-in-Memory Macro for Advanced AI Edge Chips," *IEEE Journal of Solid-State Circuits*, 2024.

[14] W.-S. Khwa *et al.*, "14.2 A 16nm 216kb, 188.4 TOPS/W and 133.5 TFLOPS/W Microscaling Multi-Mode Gain-Cell CIM Macro Edge-AI Devices," in *2025 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 68. IEEE, 2025, pp. 1–3.

[15] F. Tu *et al.*, "MulTCIM: Digital Computing-in-Memory-Based Multimodal Transformer Accelerator With Attention-Token-Bit Hybrid sparsity," *IEEE Journal of Solid-State Circuits*, vol. 59, no. 1, pp. 90–101, 2023.

[16] J. Lin *et al.*, "AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration," *Proceedings of Machine Learning and Systems*, vol. 6, pp. 87–100, 2024.

[17] M. Yu, D. Wang, Q. Shan, and A. Wan, "The Super Weight in Large Language Models," *arXiv preprint arXiv:2411.07191*, 2024.

[18] T. Chen *et al.*, "The Lottery Ticket Hypothesis for Pre-trained Bert Networks," *Advances in neural information processing systems*, vol. 33, pp. 15 834–15 846, 2020.

[19] E. Iofinova *et al.*, "How Well Do Sparse Imagenet Models Transfer?" in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 12 266–12 276.

[20] E. Frantar and D. Alistarh, "SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot," in *International Conference on Machine Learning*. PMLR, 2023, pp. 10 323–10 337.

[21] G. Bai, Y. Li, C. Ling, K. Kim, and L. Zhao, "SparseLLM: Towards Global Pruning for Pre-trained Language Models," *arXiv preprint arXiv:2402.17946*, 2024.

[22] T. Dettmers *et al.*, "SpQR: A Sparse-Quantized Representation for Near-Lossless LLM Weight Compression," *arXiv preprint arXiv:2306.03078*, 2023.

[23] S. Ashkboos *et al.*, "QUIK: Towards End-to-End 4-Bit Inference on Generative Large Language Models," *arXiv preprint arXiv:2310.09259*, 2023.

[24] Y. Zhao *et al.*, "Atom: Low-bit Quantization for Efficient and Accurate LLM Serving," *Proceedings of Machine Learning and Systems*, vol. 6, pp. 196–209, 2024.

[25] C. Guo *et al.*, "Olive: Accelerating Large Language Models via Hardware-friendly Outlier-Victim Pair Quantization," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–15.

[26] J. Koo *et al.*, "OPAL: Outlier-Preserved Microscaling Quantization Accelerator for Generative Large Language Models," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.

[27] C. Xue *et al.*, "Oltron: Algorithm-Hardware Co-design for Outlier-Aware Quantization of LLMs with Inter-/Intra-Layer Adaptation," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.

[28] A. Mishra *et al.*, "Accelerating Sparse Deep Neural Networks," *arXiv preprint arXiv:2104.08378*, 2021.

[29] A. Zhou *et al.*, "Learning N:M Fine-grained Structured Sparse Neural Networks From Scratch," *arXiv preprint arXiv:2102.04010*, 2021.

[30] W. Sun *et al.*, "DominoSearch: Find Layer-Wise Fine-Grained N:M Sparse Schemes From Dense Neural Networks," *Advances in neural information processing systems*, vol. 34, pp. 20 721–20 732, 2021.

[31] G. Fang *et al.*, "MaskLLM: Learnable Semi-Structured Sparsity for Large Language Models," *arXiv preprint arXiv:2409.17481*, 2024.

[32] C. Fang *et al.*, "Efficient N:M Sparse DNN Training Using Algorithm, Architecture, and Dataflow Co-Design," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 2, pp. 506–519, 2023.

[33] M. Zhang *et al.*, "FNM-Trans: Efficient FPGA-based Transformer Architecture with Full N: M Sparsity," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.

[34] F. Zhang *et al.*, "Efficient Memory Integration: MRAM-SRAM Hybrid Accelerator for Sparse On-Device Learning," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.

[35] Z.-G. Liu *et al.*, "S2TA: Exploiting Structured Sparsity for Energy-Efficient Mobile CNN Acceleration," in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2022, pp. 573–586.

[36] C. Zhang *et al.*, "DSTC: Dual-Side Sparsity Tensor Core for DNNs Acceleration on Modern GPU Architectures," *IEEE Transactions on Computers*, 2024.

[37] B. D. Rouhani *et al.*, "Microscaling Data Formats for Deep Learning," *arXiv preprint arXiv:2310.10537*, 2023.

[38] Y.-C. Lo *et al.*, "ISSA: Architecting CNN Accelerators Using Input-Skippable, Set-Associative Computing-in-Memory," *IEEE Transactions on Computers*, 2024.

[39] S. Han *et al.*, "Learning Both Weights and Connections for Efficient Neural Network," *Advances in neural information processing systems*, vol. 28, 2015.

[40] W. Wen *et al.*, "Learning Structured Sparsity in Deep Neural Networks," *Advances in neural information processing systems*, vol. 29, 2016.

[41] J. Lee *et al.*, "7.7 LNPU: A 25.3 TFLOPS/W Sparse Deep-Neural-Network Learning Processor with Fine-Grained Mixed Precision of FP8-FP16," in *2019 IEEE International Solid-State Circuits Conference-(ISSCC)*. IEEE, 2019, pp. 142–144.

[42] N.-C. Huang *et al.*, "ELSA: Exploiting Layer-wise N: M Sparsity for Vision Transformer Acceleration," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 8006–8015.

[43] Microsoft, "MX Pytorch Emulation Library," 2023. [Online]. Available: https://github.com/microsoft/microxcaling

[44] Y. N. Wu *et al.*, "Highlight: Efficient and Flexible DNN Acceleration with Hierarchical Structured Sparsity," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 1106–1120.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:45:59 UTC from IEEE Xplore. Restrictions apply.